

ساختن های داده محلی برای نگه داری انواع داده ها است که می تواند انواع مختلفی داشته باشد.

عملیات آکامری / ریاضی	حذف و اضافه	تکراری	تغییر	ترتیب
List []	✓	✓	✓ (1)	✓
Tuple ()	x	✓	x	✓
dict { k:v }	✓	✓	✓	✓
set { }	✓	x	x	x

در لیست می توان انواع داده ها را (دیتایپ های مختلف را در کنار هم قرار داد. بیش از یک نوع داده)

ex. Py `a = [1, 2, 3, 14, True, "Ahmad"]`

(2)

برای این که بتوانیم لیست رو به آرایه تبدیل کنیم حتماً باید داده هایی از یک نوع داشته باشیم.

`a = [1, 2, 3, 4]`

(3)

* لیست قابلیت ضرب شدن ندارد. ← فقط ضرب در یک عدد صحیح دارد.

`Print(a*2)`

فاقد جمع و تفریق

`[1, 2, 3, 4, 1, 2, 3, 4]` خروجی

تکرار

(4)

`a = [1, 2, 3]`

`Print(a + [2, 2, 2])` خروجی `[1, 2, 3, 2, 2, 2]`

(5)

(1)

* برای انجام عملیات با هر از حلقه استفاده کنیم مثلاً
 for in range (Len(u)) $\longrightarrow$ یخینه سب
 $u[i] * 2$ $\xrightarrow{\text{خروجی}}$ [2, 4, 6]

(6)

① Def zarb (number)
 return number * 2

$u = \text{list}(\text{map}(\text{zarb}, u))$

} map

② $\text{list}(\text{map}(\text{lambda } u: u * 2, u))$

③ $\text{list}(\text{filter}(\text{lambda } num: num \% 2 != 0, u))$

} filter

④ from function Tools import reduce

$sm = \text{reduce}(\text{lambda } u, y: u + y, u)$

} reduce

(2)

اولین مورد این است که ما توانیم یک لیستی از اشیاء را تبدیل کنیم به آرایه numpy
 $List \rightarrow np.array$

`import numpy as np`

`n = np.array([1,2,3])`

`print(n)`

`print(n+1)`

`print(n*3)`

`print(n/2)`

`print(n-4)`

آرایه‌های دارای عملیات
ریاضی هستند

در اینجا می‌توانیم یک آرایه جدید رو به روشی جدید به اسم `np.array` آرایه N جدیدی



shape	mean
ndim	min
dtype	max
size	std
	var

جبر خطی - ریاضی - آمار - پایه سازی شده است

وقتی تبدیل به numpy می‌شوند معمولا' آگومان به اسم `dtype` دارند که نوع داده را مشخص می‌کند

`z = [[1,2,3], [4,5,6]]`

`n = np.array(z, dtype=np.float32)` خروجی `float`

`print(n)`

حتماً باید از یک نوع داده باشند ← numpy

$X = \text{np.arange}(1, 11, 1, \text{dtype} = \text{np.uint8})$

$\text{Print}(X * 1.3) \xrightarrow{\text{خروجی}} \text{تبدیل به float شد}$

`import numpy as np`

`u = np.array([1, 2, 3])` → تبدیل لیست به آرایه

`Print(u.shape)`
`Print(u.ndim)`
`Print(u.dtype)`
`Print(u.size)`
`Print(u)`

اگر لیست بود اینها را ندانستیم →
مشاهده‌ی عینی

اولین درس به عنوان تکلیف

① مشاهده‌ی عینی

② تشخیص نوع داده

$\begin{bmatrix} - & - & - \\ - & - & - & - \end{bmatrix}$
(X)

از یک نوع باشند
تعداد سطرها و ستون‌ها برابر
شرایط لازم برای آرایه نامیده می‌شود

$Z = [[1, 2, 3], [2, 4, 6]]$

`Z = np.array(Z)`

`Print(Z)`

`Print(Z.dtype)`

`Print(Z.shape)`

`Print(Z.ndim)`

`Print(Z.size)`

* در پایتون برای تولید دنباله‌های عددی از تابع range استفاده می‌کنیم

range(1,11,2)

04.Py

اول مقدار اعشاری قبول نمی‌کند
→ range(1.1, 2.0, 3, 2, 3)

← مجموعه عددی نیست

(X)

* در نای پای به جای range → arange

X = np.arange(0, 10, 1)

→ درست کردن رنجی از اعداد

Y = np.arange(0, 1, 0.3)

X = np.arange(1, 11, 1, dtype = np.uint8)

X = np.arange(1, 11, dtype = np.uint8)

X = np.arange(11, dtype = np.uint8)

X = np.linspace(1, 2, 3)

* پس اگر آرایه ای داریم که با نای پای ساخته شده پس توانیم عملیات اصلی را با یک عدد ثابت انجام بدهیم.

X = np.array([[1, 2, 3], [4, 5, 6]])

Print(X*2)
Print(X/2)
Print(X+2) } scalar عملیات عددی

Print(X**2) → np.power(X, 2)

* یکی از مهم ترین دستورات reshape هست که باید به صورت یک tuple داده شود

$X = \text{np.array}([1, 2, 3], [4, 5, 6])$

$X.\text{reshape}((3, 2)) \rightarrow$ ضرب اعداد یکی در برابر

$\rightarrow \begin{bmatrix} [1, 2, 3] \\ [4, 5, 6] \end{bmatrix}$

$\downarrow$
 $\begin{bmatrix} [1, 2], \\ [3, 4] \\ [5, 6] \end{bmatrix}$

* ساخت اعداد تصادفی در نایپای (خیلی کامل)

$\text{Print}(\text{np.random.random}((3, 4))) \rightarrow$ اعداد تصادفی بین صفر و یک

$\text{Print}(\text{np.random.randint}(0, 20, 11)) \rightarrow \text{vector}$

$\text{np.random.rand}(5)$

$\text{np.random.randint}(5.5)$

$\text{np.random.randint}(0, 100)$

$\text{np.random.randint}(0, 100, 20)$

$\text{np.random.randint}(0, 100, (3, 4))$

موردی که می توانیم توی ماتریس داشته باشیم به دو بخش تقسیم می شود

عملیات ماتریس 2D انواع ماتریس

* هر آرایه ای که فقط دارای 1 (ب) سطر باشد ماتریس سطری می شود

$X = \text{np.array}([1, 2, 3, 4, 5, 6]) \rightarrow 2D$

$\text{Print}(X.\text{shape}) (1, 5)$

$X = \text{np.array}([1], [2], [3], [4], [5])$

$\text{Print}(X.\text{shape}) \rightarrow (5, 1)$

* ماتریس که فقط دارای یک ستون باشد ماتریس ستونی گفته می شود.

$X = \text{np.array}([1, 2, 3], [4, 5, 6], [7, 8, 9])$

* ماتریس که سطر و ستون های برابر داشته باشد ماتریس مربعی می گویند. (بعضی از عملیات ها فقط روی این امکان پذیر هست)

$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \rightarrow$ قطری

$\begin{bmatrix} 4 & 5 & 6 \end{bmatrix} \rightarrow$ درایه

$\begin{bmatrix} 7 & 8 & 9 \end{bmatrix}$

قطر اصلی

انواع ماتریس ها:

(squared) $X = \text{np.random.randint}(1, 10, (4, 4)) \rightarrow$ مربعی

(Diagonal) $X = \text{np.diag}([1, 2, 3]) \rightarrow$ قطری

(Identity) $X = \text{np.eye}(5) \rightarrow \text{Identity} \rightarrow$ همانی

$X = \text{np.zeros}((3, 4)) \rightarrow$ همه صفر

$X = \text{np.ones}((3, 4)) \rightarrow$ همه یک

... $X = \text{np.array}([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]) \rightarrow$ متغیر قطری

Print(x)
Print(x.shape) → (2,3)
→ [[1,2,3],
[4,5,6]]

استفاده کنیم ~~ravel~~ برای تبدیل به بردار باید از

Print(np.~~ravel~~(x)) → [1 2 3 4 5 6]

72

29

6

np.rv(a) → بلا صلی
np.tri(a) → جایی صلی

(26)

(12. py)

عملیات ماتریس: ضرب ضلعی (m)

X1 = np.arange(1, 13).reshape((3, -1)) (28) (28)

X2 = np.random.randint(1, 13, (3, 4))

ع. Print(a + y) → جمع دایره دایره

Print(a + 3)

(29)

Print(a - y) → دایره دایره

ضرب دایره دایره

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \cdot \begin{bmatrix} 2 & 13 \\ 17 & 5 \\ 17 & 5 \end{bmatrix} = \begin{bmatrix} - \\ - \end{bmatrix}$$

2x3 ضرب اولی = 3x1 سطر دایره

ضرب دایره

1x a + 2x b + 3x c

4x a + 5x b + 6x c

(30) multiplication

ضرب

Product

حاصل ضرب

scores = np.array([17, 18, 20], [16, 14, 13], [12, 15, 17])

coef = np.array([[10], [3], [6]])

np.dot(scores, coef)

(11) scores.dot(coef)

* یک نکته واجب reshape کردن

```
X = np.array([[1,2,3],[4,5,6]])
```

$X.reshape((1,6)) \rightarrow$ سطری $\rightarrow X.reshape((-1,-1))$

$* np.ravel(m) \rightarrow$ بردار

$X.reshape((6,1)) \rightarrow$ ستونی $\rightarrow X.reshape((-1,1))$

هر تعدادی که می شود

$x.ravel(x) \rightarrow$ vector

$x.ravel(x).reshape(1,-1) \rightarrow$ آرایه

27

10.Py

```
x = np.array([1,2,3,4,5])
```

```
print(x[::-1])  $\rightarrow$  reverse
```

```
print(x[-1])  $\rightarrow$  last index
```


- حالا باید یک سری عملیات های پایه ای در حالت سین هارایسیم که بتونیم یک سری کارها انجام بدیم.

08. py

```
X = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) astype(np.float32)
```

numpy $G_0 = I$ $G_1 = \frac{1}{2} G_0$ $G_2 = \frac{1}{4} G_1$

0 1 2 3

0 $\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$

 Slice (بدش)

1. $[5 \ 6 \ \textcircled{7} \ 8]$

② $\lim = 2$

$$2 \quad [9 \ 12 \ 14 \ 16]$$

X [دستون، سطر]

$$X[\underbrace{\text{start:stop-1:step}}_{\text{شماره}}, \underbrace{\text{start:stop-1:step}}_{\text{سیر}}]$$

`Print(X[1,2])` $\rightarrow 7$

`print(x[0:2:1, 0:3:2])` → $\begin{bmatrix} 1 & 3 \\ 5 & 7 \end{bmatrix}$

```
Print (x[:2:1, :3:2])
```

`Print(X[:, :2])` → همه سطرها و ۲ ستون را
نکته: ۲ ستون

`Print(x[:, :])` $\rightarrow$ step 2

`Print (x[:2:])` → سلسلہ 0، 1 اور 2 ستریں

⑨ $\log_{10}(x)$

آمار علمی است که بررسی روی داده ها و وقایع می پردازد بر اساس یک سری شاخص های کمی
که می تواند یک دید کلی مهبون بده.

min

کمینه

max

بیشینه

sum

جمع

(31)

14. Py

نشان دانه min , max

scores = np.array([[17, 18, 19],
[16, 14, 13],
[12, 15, 17]])

scores.min()

scores.max()

scores.min(axis=0) → np.min(scores, axis=0) axis = 0 نشان دانه

scores.max(axis=0)

axis = 1 سطرها

[12 14 13]

[17 18 19]

(32)

mean

میانگین

Print(scores.mean)(axis=1)

این از این شاخص ها در مقابل داده های پرت و داده های گسسته آسیب پذیر هستند و

15. RY

یعنی نیست.

```
X = np.array([1,2,3,4,5])
```

```
Print(np.mean(m)) → 3,0
```

```
Print(np.median(m)) → 3,0
```

35

→ finance.yahoo.com

2 شاخص های پالانسی از مالدی:

```
import yfinance as yf
```

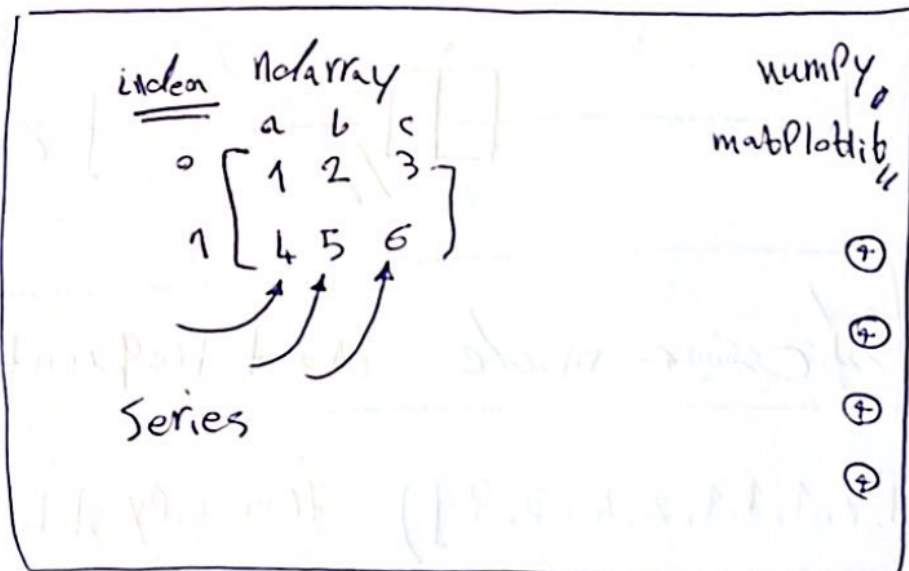
```
data = yf.download("BTC-USD", start="2020-12-17", end="2024-01-10")
```

```
Print(data)
```

```
Print(type(data))
```

Pandas

DataFrame



1. شاخص های مودری:

$$\frac{\sum a_i}{n} \leftarrow \text{حسابی (معدل)}$$

$$\frac{10 \times 2 + 30}{3} \leftarrow \text{هندسی}$$

$$\frac{3}{\frac{1}{1} + \frac{1}{2} + \frac{1}{40}} \leftarrow \text{هارمونیک}$$

میانگین

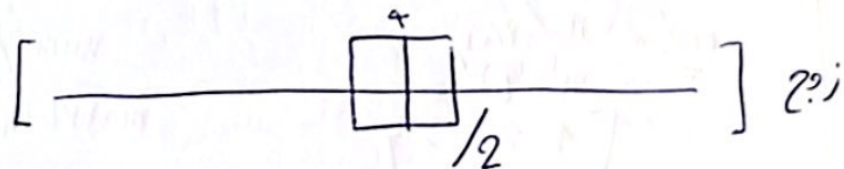
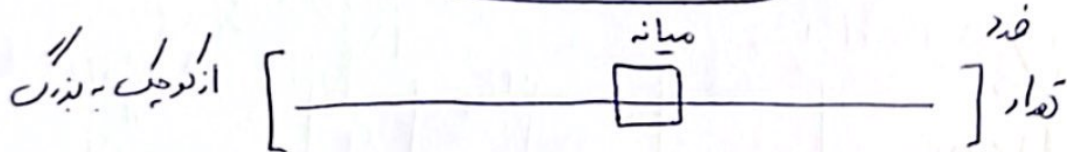
5	2	4
ریاضی	زبان	علوم
15	18	17

وزن دار

33

$$\frac{\sum a_i \cdot w_i}{\sum w_i} = \frac{15 \times 5 + 18 \times 2 + 17 \times 4}{11}$$

میان (median)



34

mode Most Frequent ← بیشترین تکرار

mod([1, 2, 1, 1, 1, 2, 4, 3, 8]) from scipy.stats import mode

mode = (1, 3)

13

CSV

xml

xlsx

JSON

.read_csv

.read_xml

.read_xlsx

.read_json

طای اسنادار.

18.Ry import pandas as pd

df = pd.read_csv("stat.csv")

print(type(df)) excel

print(df.values) json

print(type(df.values)) → ndarray (numpy)

df.to_excel("stat2.xlsx")

df.info()